



Módulo 4

Seguridad y Concerns Transversales

Curso: Terraform Hands-On (GCP)

3 h (180 min)

Objetivos de Aprendizaje

- Diseñar el modelo de **service accounts** para cada tier de AppLocker (backend, middleware, locker) siguiendo el principio de privilegio mínimo.
- Asignar **roles IAM predefinidos** (no basic roles) y crear **custom roles** solo cuando ningún rol predefinido cubra la necesidad.
- Integrar **Secret Manager** para credenciales de BD, consumiéndolas desde Terraform (data source) y desde las VMs en runtime (metadata + Workload Identity).
- Aplicar **labels obligatorios** (`env`, `app`, `tier`, `team`, `managed-by`, `cost-center`) con `locals` + `merge`.
- Configurar **budgets por label** y alertas operativas en Cloud Billing.
- Habilitar **backups automatizados** y **PITR** en Cloud SQL, más snapshots de disco en el MIG de backend.
- Detectar y responder a **drift** entre Terraform y la realidad usando `terraform plan -detailed-exitcode`.

Encuadre del módulo

Estado al llegar a M4 + mensaje clave

Estado al llegar a M4 (recap rápido)

El alumno ya tiene desplegado en su proyecto:

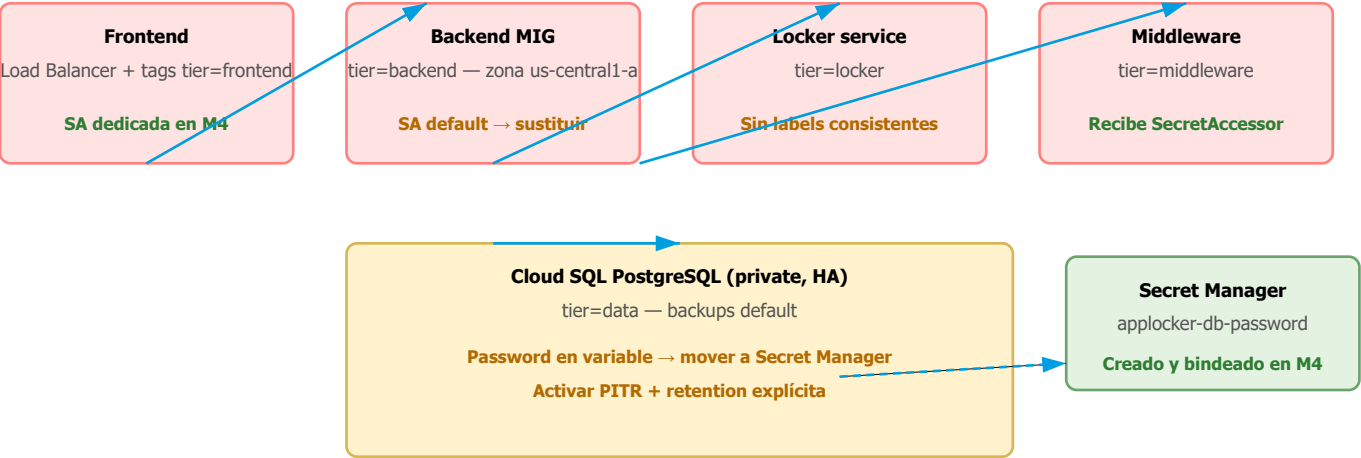
- **M1:** bucket de state remoto `applocker-tf-state-<sufijo>` en `us-central1`, versionado, con locking y workspaces `dev` / `prod`.
- **M2:** módulo `cloudsql` publicado en el Private Registry (GCS) y consumido desde `envs/dev` y `envs/prod`.
- **M3:** VPC 3-tier con subnets `frontend`, `backend`, `data`; Cloud NAT; MIG de backend con health check y autohealing; instancia Cloud SQL PostgreSQL privada con HA regional; firewall segmentado por tags.

Importante: en M4 **NO se crea infraestructura nueva de red ni de cómputo**. Se ENDURECE la existente.

El caso AppLocker — punto de partida

AppLocker 3-tier — Estado al llegar a M4

Anotaciones en rojo: lo que endurecemos en este módulo



Acciones M4 (endurecer, NO crear infra nueva)

- Crear 3 SAs dedicadas (backend / middleware / locker) y asignar roles con privilegio mínimo
- Crear secreto **applocker-db-password** y bindear a las SAs que lo necesitan
- Adjuntar SAs al MIG en lugar de la default SA
- Aplicar labels obligatorios con **locals + merge** en TODOS los recursos
- Activar backups + PITR en Cloud SQL y snapshot schedule en MIG de backend

Mensaje clave

La seguridad no se añade al final — **se aplica sobre lo que ya funciona.**

- M3 dejó la infra funcionando; ahora le damos identidad, secretos, etiquetas y disciplina operativa.
- El pipeline de CI/CD de M5 operará sobre esta base — necesita credenciales auditables, no un Owner de proyecto.
- Cuatro bloques teóricos (IAM, Secretos, Labels, Backups/Drift) se aplican de forma integrada en el lab.

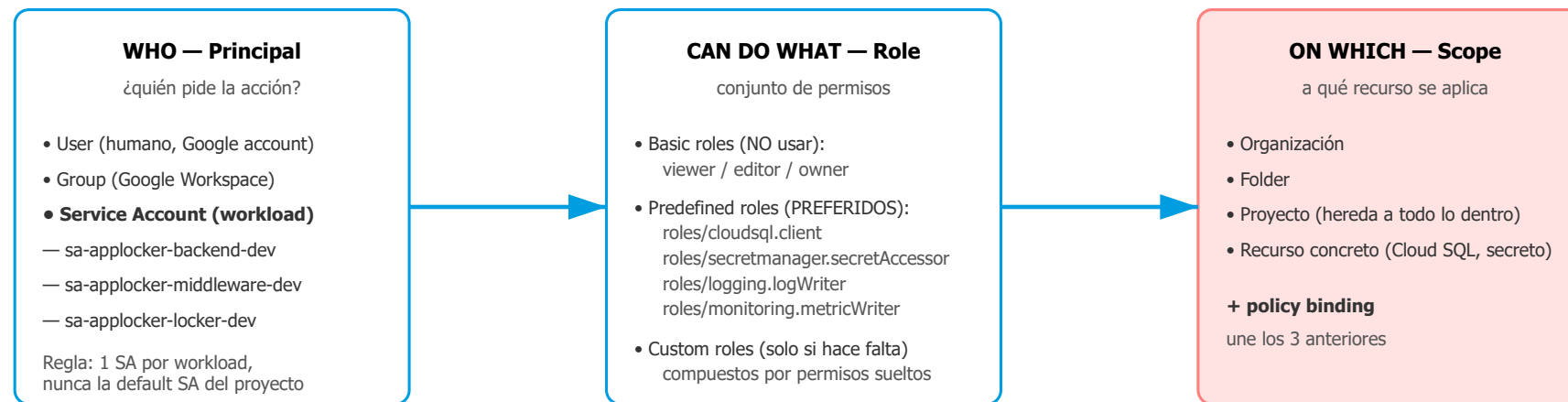
1. Teoría — IAM y Service Accounts

Quién puede hacer qué, sobre qué recurso

1.1 Modelo IAM (Who / Can do what / On which resource)

Modelo IAM — Who / Can do what / On which resource

Una policy binding une un principal, un rol y un scope



Privilegio mínimo = el role solo concede los permisos que el workload NECESITA, ni uno más.

Si un tier solo lee secretos → solo **secretAccessor**; nunca editor ni owner.

- **Principal:** quién pide la acción (user, group, service account).
- **Role:** conjunto de permisos (collection de `service.resource.verb`).
- **Policy binding:** une principal + role + scope (org, folder, proyecto o recurso).
- **Herencia:** un binding a nivel de proyecto aplica a todos los recursos debajo.

1.2 Service Accounts: identidades para workloads

- Una **service account (SA)** es una identidad no humana que usan las VMs, las pipelines o los servicios de GCP.
- Diferencia con un usuario: la SA **no tiene contraseña**; se autentica con claves (legacy), con adjunción a recursos (GCE/GKE/Cloud Functions) o con **Workload Identity Federation**.

Buenas prácticas básicas:

- Una SA por workload, no una SA compartida para todo el proyecto.
- Nombre descriptivo: `sa-<app>-<tier>-<env>` → `sa-applocker-backend-dev`.
- Nunca usar la **default Compute Engine service account** (`<project>-compute@developer.gserviceaccount.com`).
- Bloquear `iam.serviceAccountKeyCreation` a nivel de organización en producción.

1.3 Principio de privilegio mínimo

- **Regla de oro:** si un recurso solo necesita leer de GCS, el rol es `roles/storage.objectViewer` — nunca `roles/editor` ni `roles/owner`.
- **Basic roles** (`viewer`, `editor`, `owner`): demasiado amplios, no respetan privilege boundary.

Predefined roles: granularidad por servicio. Ejemplos para AppLocker:

- `roles/cloudsql.client` — abrir conexiones a Cloud SQL.
 - `roles/compute.instanceAdmin.v1` — administrar MIGs.
 - `roles/secretmanager.secretAccessor` — leer versiones de un secreto.
 - `roles/logging.logWriter` — escribir logs.
 - `roles/monitoring.metricWriter` — escribir métricas custom.
- **Custom roles:** solo cuando ningún rol predefinido cubra exactamente lo necesario. Se construyen sobre una lista de permisos (`iam.permissions.list`) y se versionan.

1.4 Anti-patrones a evitar

- Asignar `roles/owner` a una SA "para que funcione y luego ajustamos".
- Reutilizar la misma SA para `backend`, `middleware` y `locker`: si una VM se compromete, el atacante tiene los 3 roles.
- Crear claves de SA (`gcloud iam service-accounts keys create`) y guardarlas en disco o en el repo: esto es lo que precisamente tenemos que evitar; usamos Workload Identity / metadata server.



[Ref. oficial IAM — understanding roles](#)

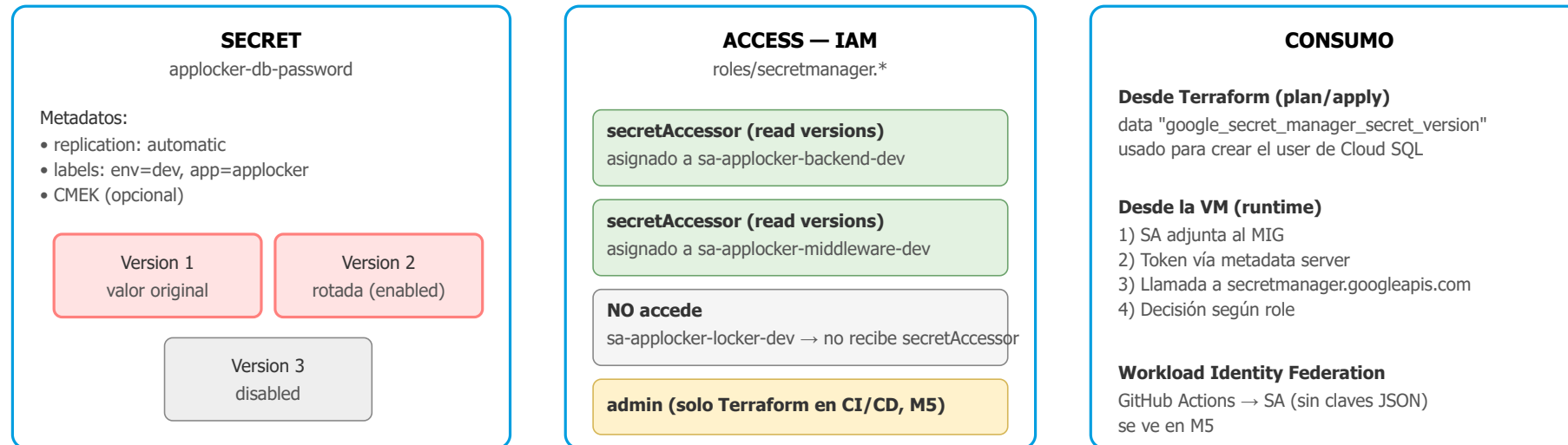
2. Teoría — Secret Manager

Secret / Version / Access

2.1 Modelo Secret / Version / Access

Secret Manager — Secret / Version / Access

Un secreto es un contenedor con versiones inmutables; el acceso se controla por IAM



Regla: secretos nunca en variables.tf, nunca en el state sin **sensitive = true**.

- **Secret:** contenedor lógico con metadatos (nombre, etiquetas, política de replicación, CMEK opcional).
- **Version:** cada `add-version` crea un valor inmutable (`projects/.../secrets/<name>/versions/N`).
- **Access:** se controla por IAM sobre el recurso `secret` (administración) y sobre cada `version` (consumo).
- Replicación: `automatic` (multi-región) o `user-managed` (regiones explícitas). Para datos europeos solemos fijar `us-central1` + `eu-west4` o automática en EU.

2.2 Rotación y CMEK

- **Rotación manual:** añadir una nueva versión con `data` distinto, marcar la antigua como `disabled`.
- **Rotación con Cloud Functions / Cloud Run Jobs** (avanzado, no se cubre en el lab): programar la rotación y actualizar el secreto desde la pipeline.
- **CMEK (Customer-Managed Encryption Key):** opcional. Cifra el secreto con una clave de Cloud KMS controlada por nosotros. Aumenta la superficie de auditoría; útil para secretos muy sensibles.

2.3 Consumo desde Terraform vs desde VMs

Desde Terraform (tiempo de plan/apply):

```
data "google_secret_manager_secret_version" "db_password" {  
  secret = "applocker-db-password"  
}
```

Útil para generar el `CREATE USER` de Cloud SQL al inicio.

Desde la VM en runtime (recomendado para credenciales rotadas):

- Adjuntar la SA `sa-applocker-backend-dev` al MIG (vía `service_account` en `google_compute_instance_template`).
- La VM obtiene un token OAuth vía metadata server (`http://metadata.google.internal/computeMetadata/v1/...`) o vía las **client libraries** que ya lo hacen automáticamente.
- Llamar a `secretmanager.googleapis.com` con ese token → el rol `roles/secretmanager.secretAccessor` decide si pasa.

Workload Identity Federation (GKE / GitHub Actions) se ve en M5.

2.4 Buenas prácticas

- Secretos **nunca** en `variables.tf`, en el state ni en outputs sin `sensitive = true`.
- Asignar `secretAccessor` solo a la SA que **realmente** consume el secreto, no a la SA del proyecto.
- Versionar siempre; no machacar la `latest`.
- Auditar accesos con Cloud Audit Logs (`data_access`).

 [Ref. oficial Secret Manager](#)

3. Teoría — Labels y costes

Visibilidad operativa y de gasto

3.1 Por qué importan los labels

- Los **labels** son pares `key=value` que se adjuntan a casi todos los recursos de GCP (instancias, buckets, instancias de Cloud SQL, discos...).

Sirven para:

- **Cost reporting:** agrupar el gasto por `team`, `app`, `env`, `cost-center`.
 - **Filtrado y operaciones:** `gcloud compute instances list --filter='labels.team=platform'`.
 - **Políticas:** `iam_condition` con `resource.matchLabels(...)` para abrir puertas solo a recursos etiquetados.
- **Límites:** máximo 64 labels por recurso, claves con regex `[a-z][a-z0-9_-]{0,62}`.

3.2 Labels obligatorios del curso

Para AppLocker se establecen como **contrato**:

Key	Ejemplo	Obligatorio	Propósito
env	dev, prod	Sí	Entorno
app	applocker	Sí	Aplicación
tier	backend, middleware, locker, data	Sí	Capa lógica
team	platform-mm	Sí	Equipo responsable
managed-by	terraform	Sí	Herramienta de provisionamiento
cost-center	CC-1042	Sí	Cuenta de coste interna

Si un recurso no tiene estos 6 labels, el módulo (o el alumno) está incompleto.

3.3 `locals` + `merge` para aplicar masivamente

Patrón recomendado para no repetir los 6 labels en cada recurso:

```
locals {
  common_labels = {
    app      = "applocker"
    env      = var.env
    team     = "platform-mm"
    managed-by = "terraform"
    cost-center = "CC-1042"
  }
}

resource "google_compute_instance_template" "backend" {
  # ...
  labels = merge(
    local.common_labels,
    {
      tier = "backend"
    }
  )
}
```

Para recursos anidados (p.ej. discos de un instance template) se repite el merge manualmente.

3.4 Budgets y alertas

- **Cloud Billing Budgets** permite definir un presupuesto mensual con **umbrales** (50%, 90%, 100% del presupuesto).
- **Filtros por label:** crear un budget específico para `labels.app=applocker` y otro por `labels.cost-center=CC-1042`.
- **Notificaciones:** email o Pub/Sub topic (este último permite reaccionar automáticamente, p.ej. cortar despliegues).
- Frecuencia de alertas: mensual (al cierre) o por umbral.

 [Ref. oficial Labels](#)

4. Teoría — Backups, recovery y drift detection

Disciplina operativa

4.1 Backups automatizados en Cloud SQL

- **Backups automáticos:** `settings.backup_configuration.enabled = true` . Se ejecutan diariamente en la `start_time` indicada.
- **Retention:** `settings.backup_configuration.backup_retention_settings { retained_backups = N }` . Para PITR hace falta `transaction_log_retention_days` .
- **Point-in-Time Recovery (PITR):** con los logs de transacciones, podemos restaurar a cualquier segundo dentro de la ventana de retención.
- **Location:** por defecto en la misma región; se puede forzar `location = "eu"` para multi-región EU.

4.2 Snapshots de disco y clones

Para los discos del MIG de backend (Compute Engine):

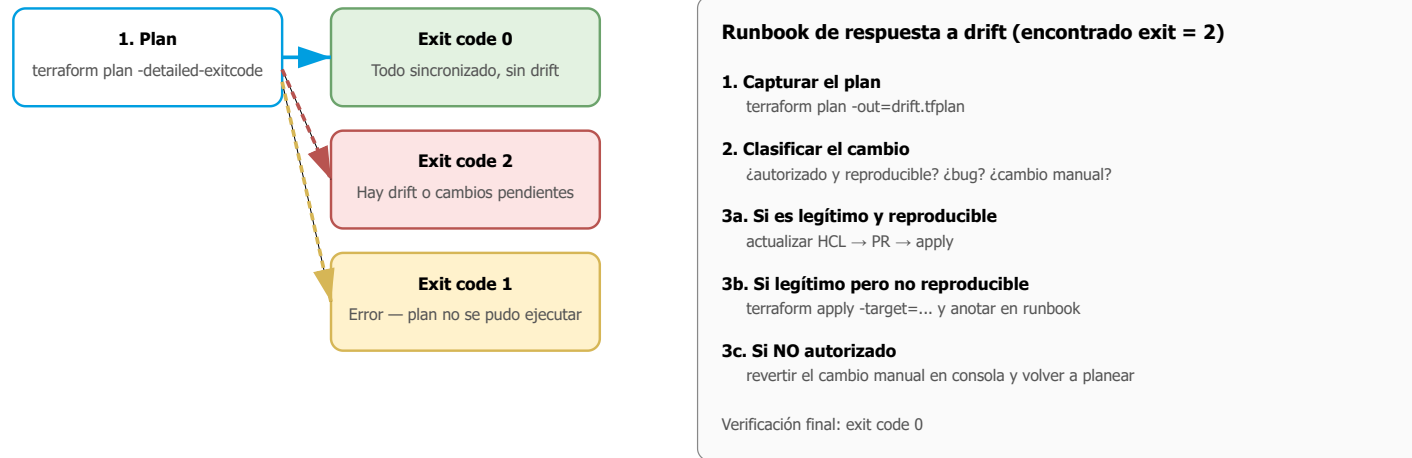
- **Snapshot estándar:** `google_compute_snapshot`, programado o manual.
- **Snapshot schedule:** `google_compute_resource_policy` con tipo `SNAPSHOT_SCHEDULE` (diario/semanal) y retention.

Clone de Cloud SQL para entornos efímeros: `google_sql_database_instance.clones` (no entra en el lab, solo se nombra).

4.3 Drift detection con Terraform

Drift detection — terraform plan -detailed-exitcode

Si la realidad diverge del state, Terraform te lo dice con un exit code



- **Drift** = la realidad diverge del state. Causas típicas: cambios manuales por consola, autoscaling añade nodos, scripts fuera de Terraform.
- `terraform plan`: comando base. Devuelve `0` si no hay cambios, `1` si hay error, `2` si hay cambios pendientes.
- `terraform plan -detailed-exitcode`: además de los códigos anteriores, devuelve:
 - `0` → sin cambios (infra sincronizada).
 - `1` → error.
 - `2` → hay cambios pendientes (drift o cambios en HCL pendientes de aplicar). En CI/CD (M5) se usa para **bloquear merges**.
- `terraform refresh` ya no es necesario: `plan` reconcilia el state contra la realidad en cada ejecución.

4.4 Runbook de respuesta a drift

1. `terraform plan -out=drift.tfplan .`
2. Inspeccionar el diff. Clasificar el cambio: ¿autorizado? ¿bug? ¿cambio manual de un dev?
3. Si autorizado y reproducible → actualizar el HCL, hacer PR, aplicar.
4. Si autorizado pero no queremos reproducirlo (p.ej. un script de remediación) → `terraform apply -target=...` y dejar constancia en el runbook.
5. Si no autorizado → revertir el cambio manual desde consola, después `terraform plan` y verificar exit code 0.



[Ref. oficial Cloud SQL Backup/Recovery](#)

5. Lab — Hardening de AppLocker

~110 min — guiado por el formador

5.0 Prerrequisitos y APIs

- Repositorio del M3 intacto: `infra/envs/dev` con la raíz del proyecto AppLocker.
- State remoto del M1 funcionando en `gs://applocker-tf-state-<sufijo>`.
- Workspace activo: `terraform workspace select dev`.
- Permisos: `roles/iam.serviceAccountAdmin`, `roles/secretmanager.admin`, `roles/billing.admin` (solo para el budget).

```
gcloud services enable \  
  iam.googleapis.com \  
  secretmanager.googleapis.com \  
  cloudbilling.googleapis.com
```

Tip: si una API ya está habilitada, el comando no falla.

5.1 Crear service accounts por tier

Bloque HCL en `infra/modules/iam/main.tf` (nuevo módulo introducido en este módulo):

```
locals {
  tiers = toset(["backend", "middleware", "locker"])
}

resource "google_service_account" "tier" {
  for_each = local.tiers

  account_id    = "sa-applocker-${each.key}-${var.env}"
  display_name  = "AppLocker ${title(each.key)} (${var.env})"
  description   = "SA dedicada al tier ${each.key} del entorno ${var.env}."
  project       = var.project_id
}
```

Verificar tras aplicar:

```
gcloud iam service-accounts list \
  --project=${TF_VAR_project_id} \
  --filter="email~'sa-applocker-.*-${TF_VAR_env}'"
```

5.2 Asignar roles con privilegio mínimo

En el mismo módulo `iam`, declaramos los bindings:

```
data "google_project" "project" {
  project_id = var.project_id
}

# Roles para TODAS las tiers: solo lo mínimo para escribir logs y métricas
resource "google_project_iam_member" "tier_logging" {
  for_each = local.tiers

  project = data.google_project.project.project_id
  role    = "roles/logging.logWriter"
  member  = google_service_account.tier[each.key].member
}

resource "google_project_iam_member" "tier_monitoring" {
  for_each = local.tiers

  project = data.google_project.project.project_id
  role    = "roles/monitoring.metricWriter"
  member  = google_service_account.tier[each.key].member
}
```

5.3 Verificación de bindings

Aplicar y verificar:

```
terraform apply -target=module.iam

for sa in backend middleware locker; do
  echo "=== sa-applocker-${sa}-${TF_VAR_env} ==="
  gcloud projects get-iam-policy ${TF_VAR_project_id} \
    --flatten="bindings[].members" \
    --filter="bindings.members:serviceAccount:sa-applocker-${sa}-${TF_VAR_env}@" \
    --format="table(bindings.role)"
done
```

Decisión deliberada: si el alumno pregunta "¿por qué el `locker` no tiene acceso a SQL?" — porque la regla del curso es que **cada tier solo abre lo que usa**. Si en el futuro el `locker` necesita SQL, se le añade el binding en un PR aparte.

5.4 Vincular SAs al MIG y a Cloud SQL

Modificar el `instance_template` del backend (y replicar para `middleware` y `locker`):

```
resource "google_compute_instance_template" "backend" {
  name_prefix = "applocker-backend-tmpl-${var.env}-"
  machine_type = "e2-medium"
  region      = var.region

  service_account {
    email = module.iam.service_accounts["backend"].email
    scopes = ["https://www.googleapis.com/auth/cloud-platform"]
  }

  # ... network, disks, metadata, labels (ver 5.6) ...
}
```

Para Cloud SQL (vinculación vía `iam_member` ya creada en 5.2) **no** se cambia la password en `users` — el siguiente paso crea el secreto y rota.

```
gcloud compute instance-groups managed list-instances \
  applocker-backend-mig-${TF_VAR_env} \
  --region=${TF_VAR_region}
```


5.5 Crear secretos y consumir desde Terraform

Crear el secreto con un valor inicial (simula la password actual):

```
export DB_PASSWORD=$(openssl rand -base64 24)
echo -n "$DB_PASSWORD" | gcloud secrets create applocker-db-password \
  --replication-policy=automatic \
  --data-file=-
```

Advertencia del formador: el comando anterior usa `gcloud` para el bootstrap. A partir de aquí el secreto se gestiona con Terraform.

5.5b Declarar el secreto en Terraform

Importar el secreto al state (`infra/envs/dev/secrets.tf`):

```
resource "google_secret_manager_secret" "db_password" {
  secret_id = "applocker-db-password"
  project   = var.project_id

  replication {
    automatic = true
  }

  labels = merge(local.common_labels, { tier = "data" })
}

resource "google_secret_manager_secret_iam_member" "db_password_readers" {
  for_each = toset(["backend", "middleware"])

  project   = var.project_id
  secret_id = google_secret_manager_secret.db_password.secret_id
  role      = "roles/secretmanager.secretAccessor"
  member    = module.iam.service_accounts[each.key].member
}
```

5.6 Aplicar labels obligatorios con `locals` + `merge`

Definir el bloque `common_labels` en `envs/dev/locals.tf`:

```
locals {
  common_labels = {
    app      = "applocker"
    env      = var.env
    team     = "platform-mm"
    managed-by = "terraform"
    cost-center = "CC-1042"
  }
}
```

Aplicar a TODOS los recursos existentes (instance templates, MIG, instance group, health check, instance de Cloud SQL, bucket del M1, secretos, SAs):

```
resource "google_compute_instance_template" "backend" {
  # ... (resto del bloque) ...
  labels = merge(local.common_labels, { tier = "backend" })
}
```

5.6b Recordatorio: labels en recursos anidados

Los **discos** y las **instancias hijas** dentro de un `instance_template` también aceptan labels; hay que mergearlos explícitamente:

```
disk {  
  source_image = "debian-cloud/debian-12"  
  auto_delete  = true  
  boot         = true  
  
  labels = merge(local.common_labels, { tier = "backend" })  
}
```

Verificar con:

```
gcloud compute instances list \  
--project=${TF_VAR_project_id} \  
--filter="labels.app=applocker" \  
--format="table(name,zone,labels.env,labels.tier,labels.team,labels.cost-center)"
```

5.7 Habilitar backups y PITR en Cloud SQL

Modificar el módulo `cloudsql` (o pasar valores desde la raíz):

```
resource "google_sql_database_instance" "main" {
  name          = "applocker-db-${var.env}"
  project       = var.project_id
  region       = var.region
  database_version = "POSTGRES_15"
  # ... tier, network, ha ...

  settings {
    tier              = var.tier
    availability_type = "REGIONAL"
    disk_size        = 50
    disk_type        = "PD_SSD"

    backup_configuration {
      enabled              = true
      start_time           = "03:00"
      location             = "eu"
      point_in_time_recovery_enabled = true
      transaction_log_retention_days = 7
    }
  }
}
```

5.7b Snapshot schedule para el MIG de backend

```
resource "google_compute_resource_policy" "backend_snapshot" {
  project = var.project_id
  region  = var.region
  name    = "applocker-backend-snap-${var.env}"

  snapshot_schedule_policy {
    schedule {
      hourly_schedule {
        hours_in_cycle = 24
        start_time      = "04:00"
      }
    }
    retention_policy {
      max_retention_days      = 7
      on_source_disk_delete = "KEEP_AUTO_SNAPSHOTS"
    }
    snapshot_properties {
      labels = local.common_labels
    }
  }
}
```

Verificar:

```
gcloud sql instances describe applocker-db-${TF_VAR_env} \
--project=${TF_VAR_project_id} \
--format="value(settings.backupConfiguration)"
```

Debe devolver `pointInTimeRecoveryEnabled: true`, `enabled: true`, `transactionLogRetentionDays: '7'`.

5.8 Validación final

1. `terraform fmt -recursive .`
2. `terraform validate .`
3. `terraform plan -out=hardening.tfplan` → debe proponer los cambios esperados.
4. `terraform apply hardening.tfplan .`

5.8a Smoke test de drift

```
terraform plan -detailed-exitcode  
echo "Exit code: $?"
```

- `0` → sin drift. **Aprobado.**
- `2` → hay drift. Inspeccionar con `terraform plan -out=...` y revisar.

5.8b Smoke test de identidad

```
# Como SA de backend (vía impersonación)
gcloud config set account sa-applocker-backend-${TF_VAR_env}@${TF_VAR_project_id}.iam.gserviceaccount.com

# Esto debe funcionar:
gcloud secrets versions access latest --secret=applocker-db-password

# Esto debe FALLAR (locker no tiene secretAccessor):
gcloud config set account sa-applocker-locker-${TF_VAR_env}@${TF_VAR_project_id}.iam.gserviceaccount.com
gcloud secrets versions access latest --secret=applocker-db-password
# Error esperado: Permission denied
```

5.8c Smoke test de labels

```
gcloud compute instances list \
  --filter="labels.app=applocker AND NOT labels.cost-center:CC-*" \
  --format="value(name)"
# Debe devolver lista vacía
```

5.9 Limpieza del lab

No borrar la infraestructura — todavía la necesitamos en M5 y M6.

Pasos de limpieza **a nivel de código** (no de recursos):

- Revisar que ningún output expone la password (`sensitive = true` donde aplique).
- Confirmar que no quedan invocaciones de `iam.serviceAccountKey.create` .
- `git add . && git commit -m "feat(m4): harden AppLocker with IAM, Secret Manager, labels and backups" .`
- Dejar el workspace `dev` activo.

A nivel de **consola**: borrar el budget de prueba creado durante el lab si era efímero (o dejarlo y notarlo en el runbook).

6. Cierre, Q&A y preparación de M5

6.1 Resumen del módulo

- Repaso de los 7 objetivos: ¿cuáles se han tocado en el lab, cuáles quedan como referencia.
- Recordatorio: el módulo no introduce GKE/Kubernetes; el Workload Identity que se use en M5 se aplica a GitHub Actions, no a clústeres.

6.2 Q&A

- Resolver dudas concretas sobre roles, secretos, labels o backups.
- Espacio para preguntas del tipo "¿y si nuestro `locker` necesita escribir en BigQuery?" — la respuesta modelo es: **se añade un binding en un PR, no se edita a mano.**

6.3 Adelanto de M5 — CI/CD y Automatización

Lo que viene: Terraform Cloud (o alternativa OSS) + GitHub Actions + OPA.

Conexión directa con lo aprendido hoy:

- El `terraform plan -detailed-exitcode` se usará como **gate** del pipeline.
- La SA de la pipeline (M5) se autenticará vía **Workload Identity Federation** contra GitHub, sin claves JSON.
- El budget por label y las políticas IAM se pueden reforzar con OPA / Sentinel en M5.

6.4 Preparación antes de M5

- Tener una **organización de GitHub** lista (o equivalente: GitLab, Bitbucket).
- Confirmar que el bucket de state tiene **uniform bucket-level access** (para que la pipeline use una sola SA, no identidades por usuario).
- Releer la sección de Service Accounts del M4 — la pipeline de M5 reusará este modelo.

Resumen

Puntos Clave

- **Una SA por tier**, con roles predefined y nunca basic roles — esa es la frontera entre "esto escala" y "esto es un riesgo operativo".
- **Secretos en Secret Manager**, consumidos vía data source en Terraform y vía metadata server en runtime — nunca en variables ni en el state sin `sensitive`.
- **Labels obligatorios** con `locals + merge` aplicados a TODOS los recursos, incluidos los anidados; sin eso, ni budgets ni políticas IAM por etiqueta funcionan.
- **Backups automáticos + PITR** en Cloud SQL, snapshot schedule en el MIG de backend; retención explícita y location forzada a `eu` si aplica.
- **Drift detection** con `terraform plan -detailed-exitcode`: exit 0 = paz, exit 2 = hay que abrir un runbook, exit 1 = algo falló.
- **M4 endurece, M5 automatiza**: el `plan -detailed-exitcode` se convierte en gate de CI/CD y las SAs se reutilizan vía Workload Identity Federation.

Referencias

Recursos oficiales

- [IAM y roles \(oficial Google\)](#)
- [Service Accounts \(oficial Google\)](#)
- [Secret Manager \(oficial Google\)](#)
- [Secret Manager — acceso desde aplicaciones](#)
- [Labels \(oficial Google\)](#)
- [Cloud Billing Budgets](#)
- [Cloud SQL — Backup y recovery](#)
- [Cloud SQL — PITR](#)
- [Compute Engine — Snapshot schedules](#)
- [Terraform — `plan -detailed-exitcode`](#)
- [Terraform — drift detection tutorial](#)



¡Gracias!

Módulo 4 completado

Siguiente: Módulo 5 — CI/CD y Automatización